

宁波大学
NINGBO UNIVERSITY

实 验 报 告

实验六《排序算法的比较》

学生姓名 顾凯琪

学号 226001624

课程名称 数据结构与算法

实验学期 2023 至 2024 学年 第 二 学期

所在学院 信息科学与工程学院

专业班级 计算机科学与技术 2 班

任课教师 赵杰煜

实验成绩

一、实验题目

排序算法的比较

二、实验目的：

随机产生 10000 个数据，进行起泡排序、快速排序、希尔排序、归并排序、堆排序、基数排序，并比较各种算法的排序时间。

三、实验设备与环境

微型计算机、Windows 系列操作系统 、Visual Studio、Pycharm 系列软件

四、实验内容

随机产生 10000 个数据，进行起泡排序、快速排序、希尔排序、归并排序、堆排序、基数排序，并比较各种算法的排序时间。

五、概要设计（思路、算法、步骤等）

添加了希尔排序、四分归并、四叉树堆排、最小基数排序

冒泡：

思路：冒泡排序是一种简单的排序算法，通过重复遍历列表，比较相邻元素并交换它们的位置，将较大（或较小）的元素逐步“冒泡”到列表的一端。

算法：

1. 从列表的起始位置开始，依次比较相邻的两个元素。
2. 如果前一个元素大于后一个元素，则交换它们的位置。
3. 经过一轮比较后，最大（或最小）的元素将被“冒泡”到列表的末端。
4. 对剩余未排序的部分重复以上步骤，直到整个列表有序。

步骤：

1. 初始化变量 i 为 0，用于标记已经排序的元素数量。
2. 使用嵌套循环，外层循环控制遍历次数，内层循环进行元素比较和交换。
3. 内层循环每次将未排序部分中最大的元素移动到列表末尾。

4. 外层循环结束时，列表已经排序完成。

快排：

思路：快速排序是一种分治法排序算法，通过选取一个“基准”元素，将列表分为两部分，使得一部分元素小于基准，另一部分元素大于基准，然后递归地对这两部分进行排序。

算法：

1. 选择一个基准元素（通常是第一个或最后一个元素）。
2. 分区操作：将列表分为两部分，一部分元素小于基准，另一部分元素大于基准。
3. 递归地对分区后的两部分列表进行快速排序。
4. 组合排序结果。

步骤：

1. 如果列表长度小于等于 1，则返回该列表。
2. 选择基准元素 pivot。
3. 使用列表推导式或循环将列表分成两个子列表：小于基准的 left 和大于基准的 right。
4. 递归地对 left 和 right 进行排序，并将结果组合。

希尔排序：

思路：希尔排序是一种非比较排序算法，它是插入排序的改进版本。该算法通过将待排序的数组划分为若干个子序列，然后分别对每个子序列进行插入排序，最后再对整个数组进行一次插入排序。关键在于选择适当的间隔序列，通过这种方式可以实现对数组的快速排序。

算法步骤：

1. 选择一个增量序列，通常为 $n/2, n/4, \ldots, 1n/2, n/4, \ldots, 1$，其中 n 是数组

的长度。

2. 对于每个增量，从第增量个元素开始，逐个对其所在的组进行插入排序。
3. 重复上述步骤，直到增量为 1，完成最后一次插入排序。

归并：

思路：归并排序也是一种分治法排序算法，将列表分成若干子列表，每个子列表只有一个元素，然后合并子列表，使其有序，最终合并成一个有序列表。

算法：

1. 将列表分成两部分。
2. 递归地对每部分进行归并排序。
3. 合并两个有序部分，得到排序后的列表。

步骤：

1. 如果列表长度小于等于 1，则返回该列表。
2. 将列表分成两部分 left 和 right。
3. 递归地对 left 和 right 进行排序。
4. 合并排序后的 left 和 right。

堆排：

思路：堆排序是一种基于堆数据结构的排序算法，通过构建最大堆或最小堆，将堆顶元素移除并重新构建堆，直到堆为空。

算法：

1. 将列表构建成一个最大堆。
2. 将堆顶元素与末尾元素交换，并减少堆的大小。
3. 重新构建堆，重复以上步骤直到堆的大小为 1。

步骤：

1. 构建最大堆。
2. 将堆顶元素（最大值）与末尾元素交换。
3. 缩小堆的大小，并重新调整堆。
4. 重复上述过程，直到堆为空。

基数：

思路：基数排序是一种非比较排序算法，通过按位对数字进行排序，从最低位到最高位，逐位排序，最终得到有序列表。

算法：

1. 从最低有效位开始，对每个位进行计数排序。
2. 依次对每个位进行排序，直到最高位。

步骤：

1. 找到列表中最大的数字，确定排序的位数。
2. 从最低位到最高位，对每个位进行计数排序。
3. 将排序结果合并，得到最终的有序列表。

六、详细设计（核心代码、算法流程图等）

代码见附录

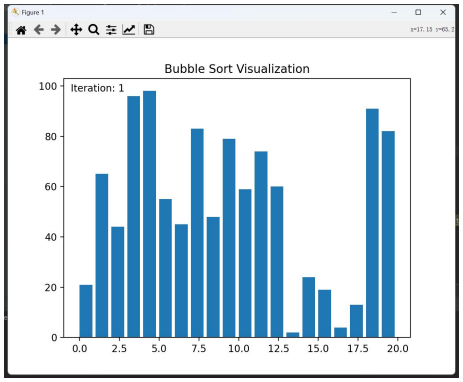
七、测试结果及分析

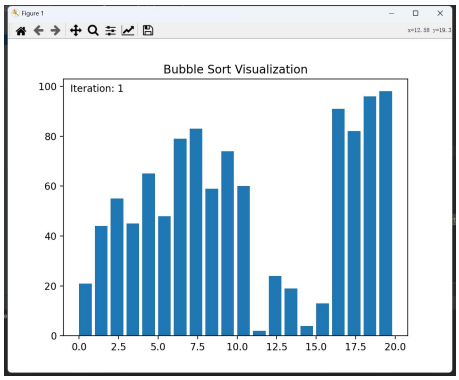
因篇幅限制，只输出前 50 个数据，原数组有 10000 个数据，可视化部分若数组内容过多导致可视化效果很差（所有条柱都黏在一起，因此只考虑 20 个数据的可视化）,排序时间十组取均值

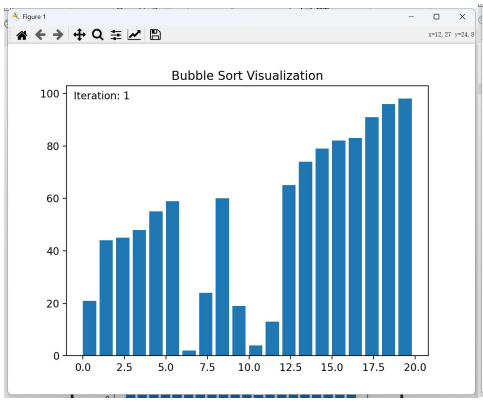
特别需要注意的是：本人在原本实验的基础上添加了希尔排序，并将二分归并改为四分归并，且堆排也改为四叉树，基数排序为最小基数排序

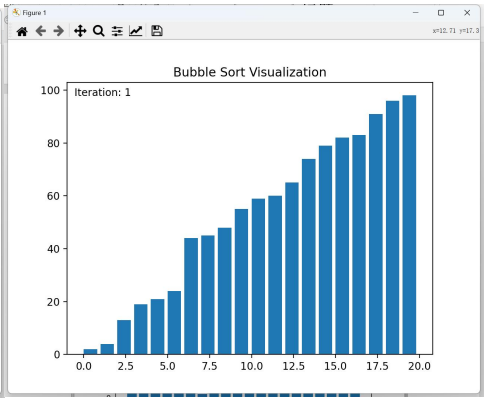
冒泡：


C:\Users\asus\anaconda3\python.exe "C:\Us
冒泡排序十次平均时间： 5.466049695014954

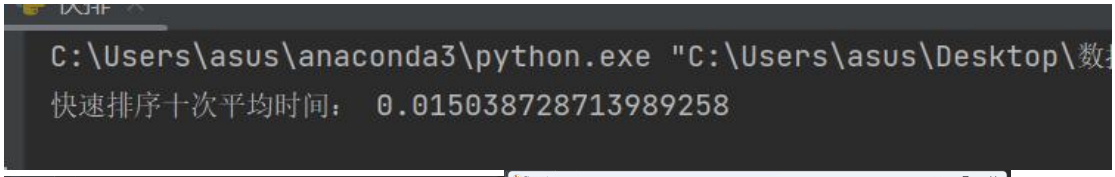

Bubble Sort Visualization
Iteration: 1

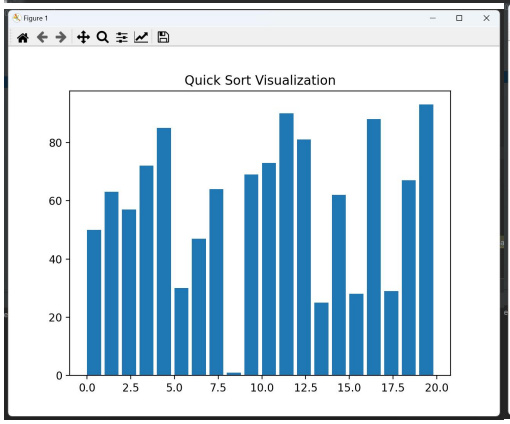

Bubble Sort Visualization
Iteration: 1

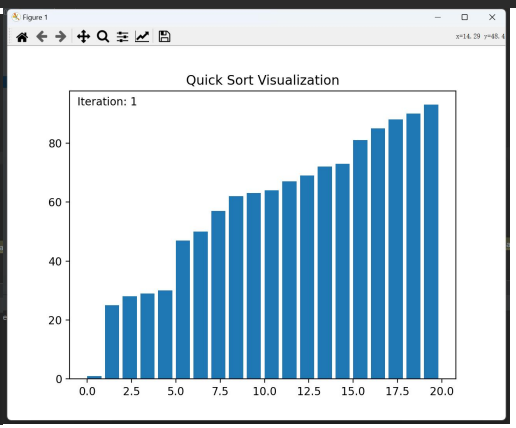

Bubble Sort Visualization
Iteration: 1


Bubble Sort Visualization
Iteration: 1

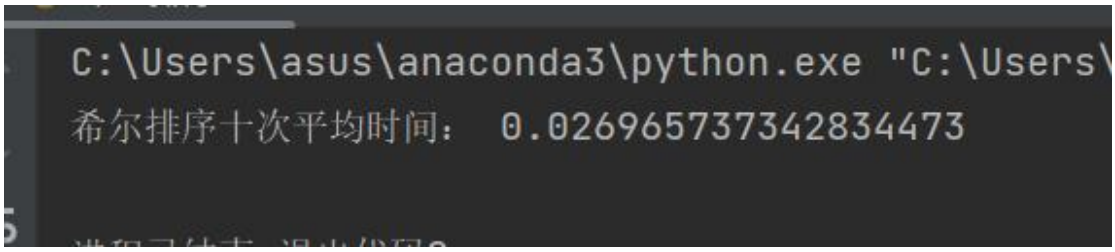
快排：

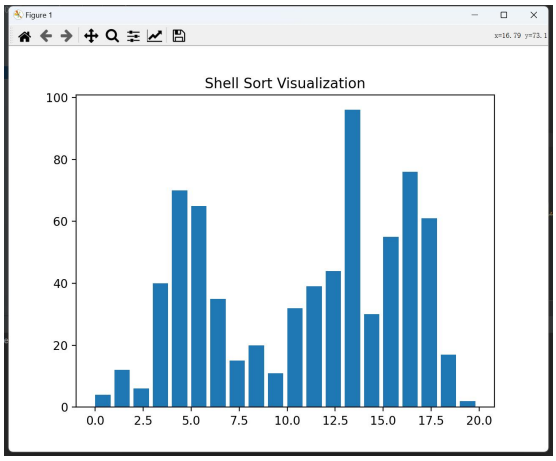

C:\Users\asus\anaconda3\python.exe "C:\Users\asus\Desktop\数
快速排序十次平均时间： 0.015038728713989258

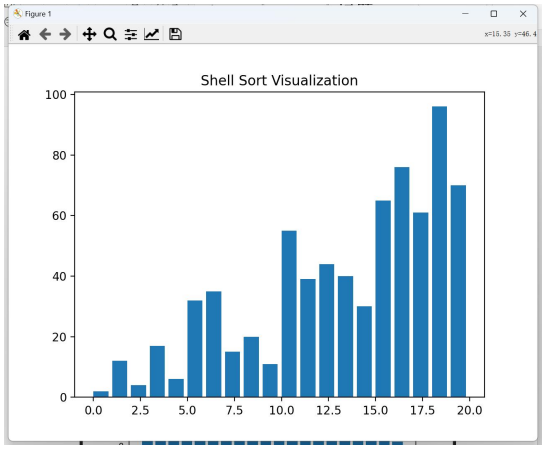

Quick Sort Visualization

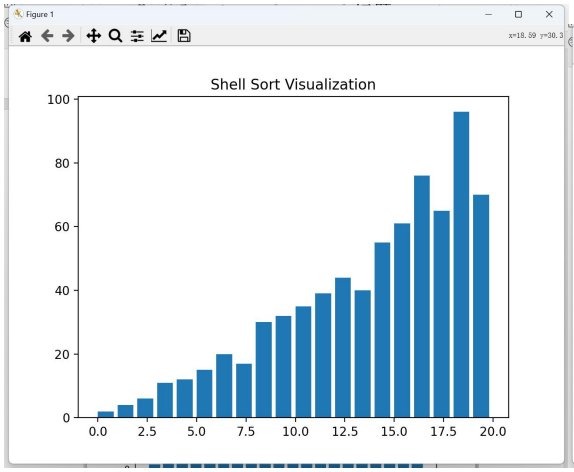

Quick Sort Visualization
Iteration: 1

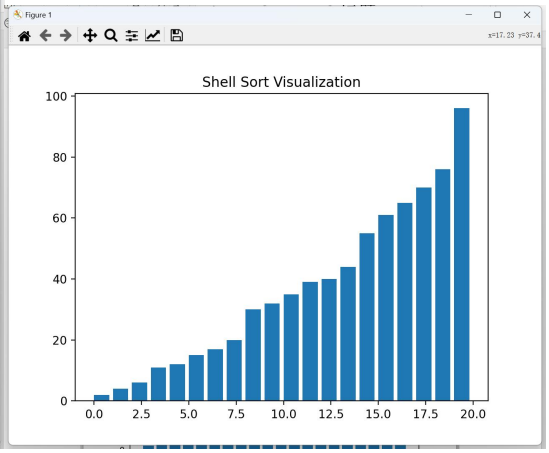
希尔排序：


C:\Users\asus\anaconda3\python.exe "C:\Users\
希尔排序十次平均时间： 0.026965737342834473


Figure 1
Shell Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

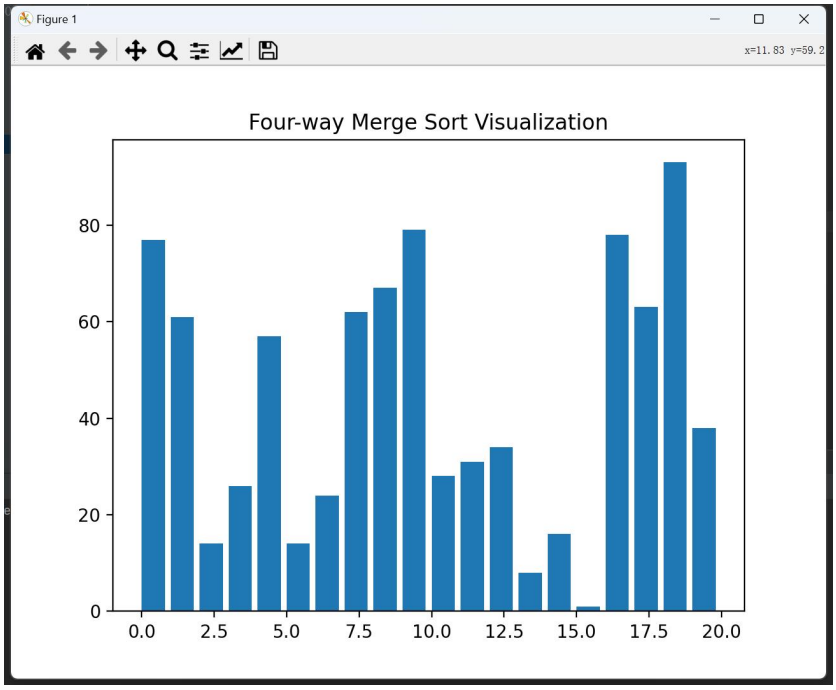

Figure 1
Shell Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

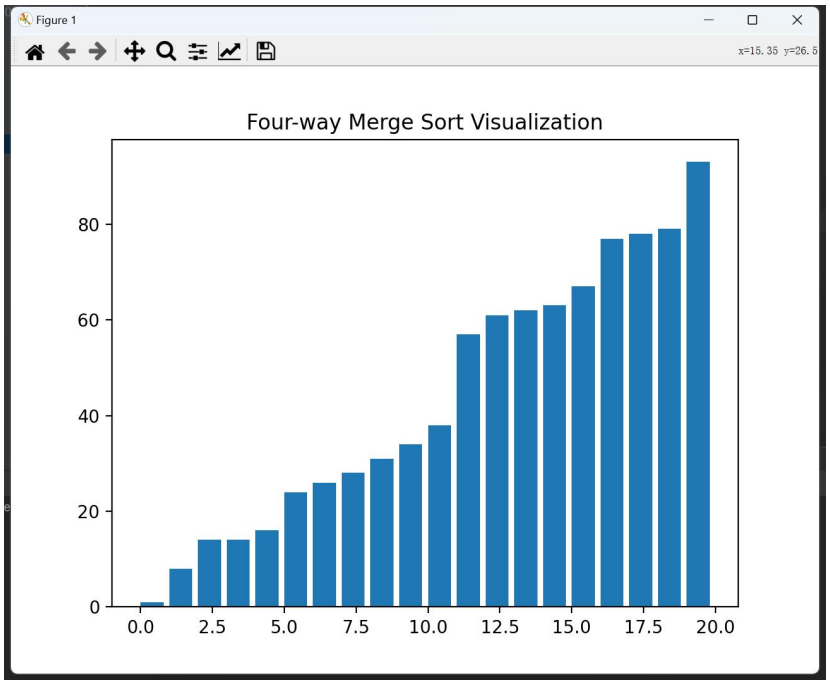

Figure 1
Shell Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Figure 1
Shell Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

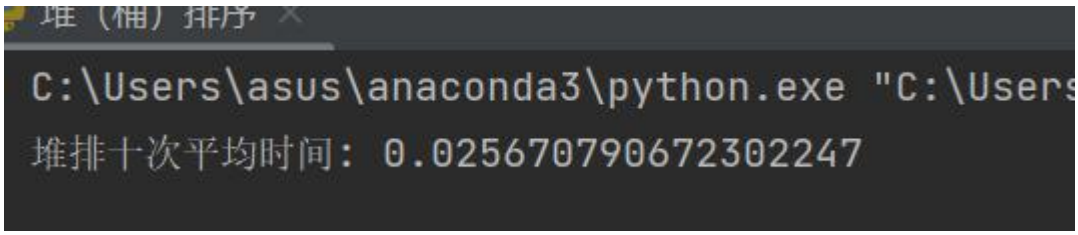
归并排序（四分）：

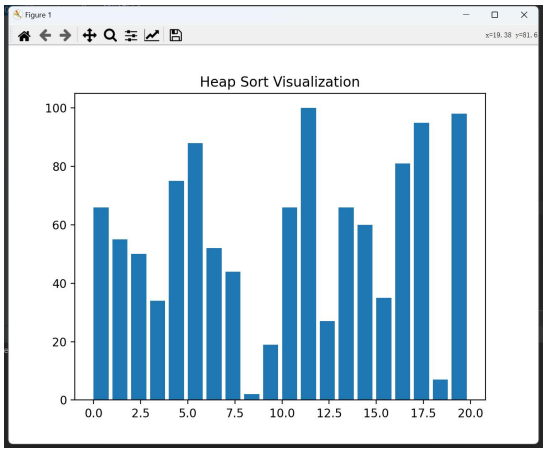
```
C:\Users\asus\anaconda3\python.exe "C:\User
四分归并排序十次平均时间：  0.022882270812988 28
```

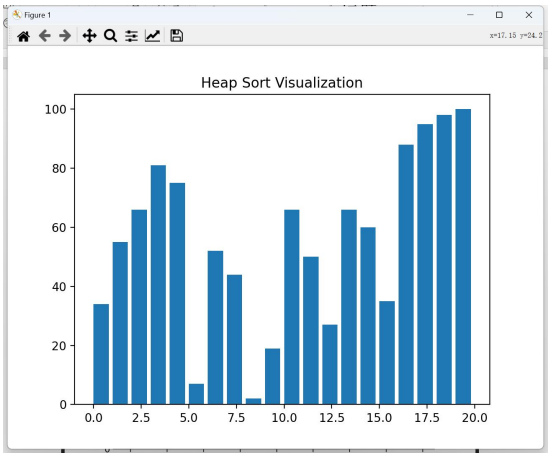


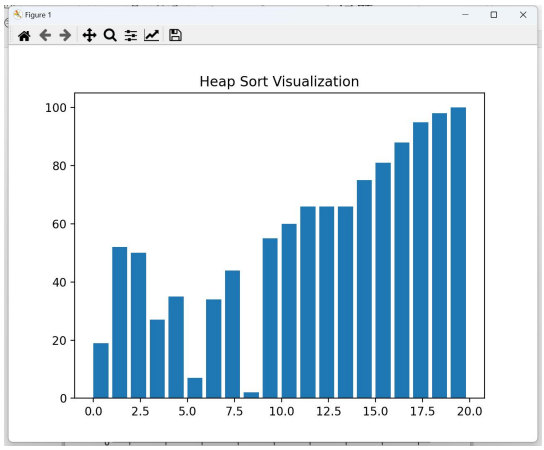


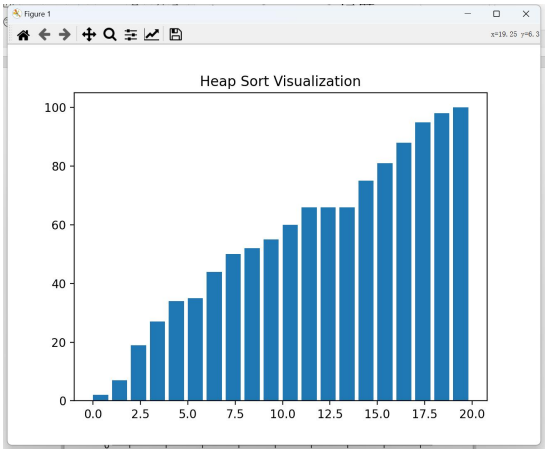
堆排序（四叉树）


C:\Users\asus\anaconda3\python.exe "C:\Users
堆排十次平均时间: 0.025670790672302247


Figure 1
Heap Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

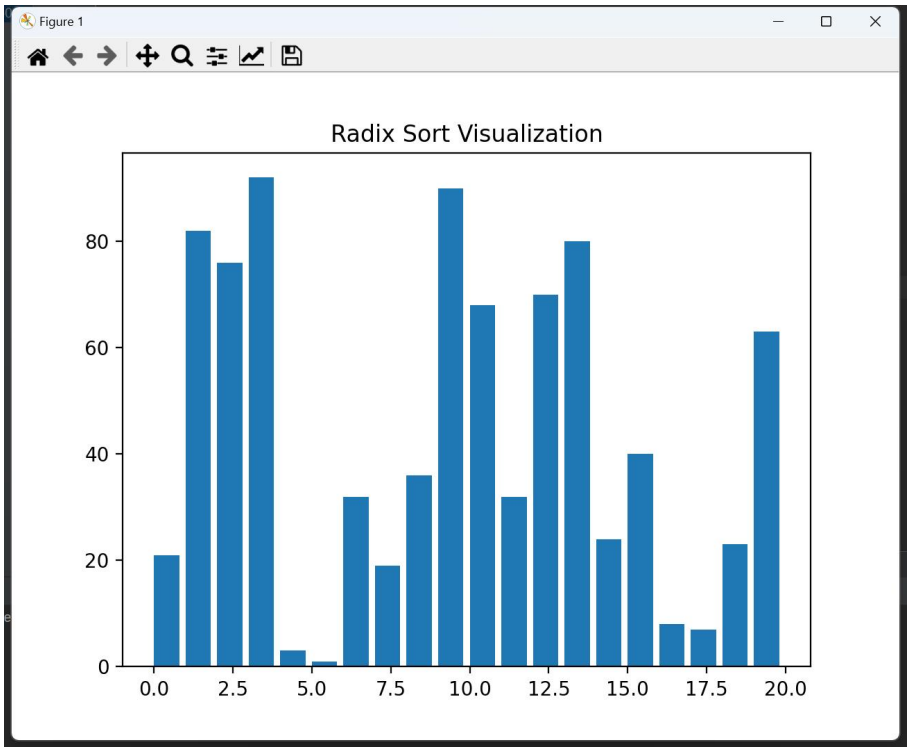

Figure 1
Heap Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

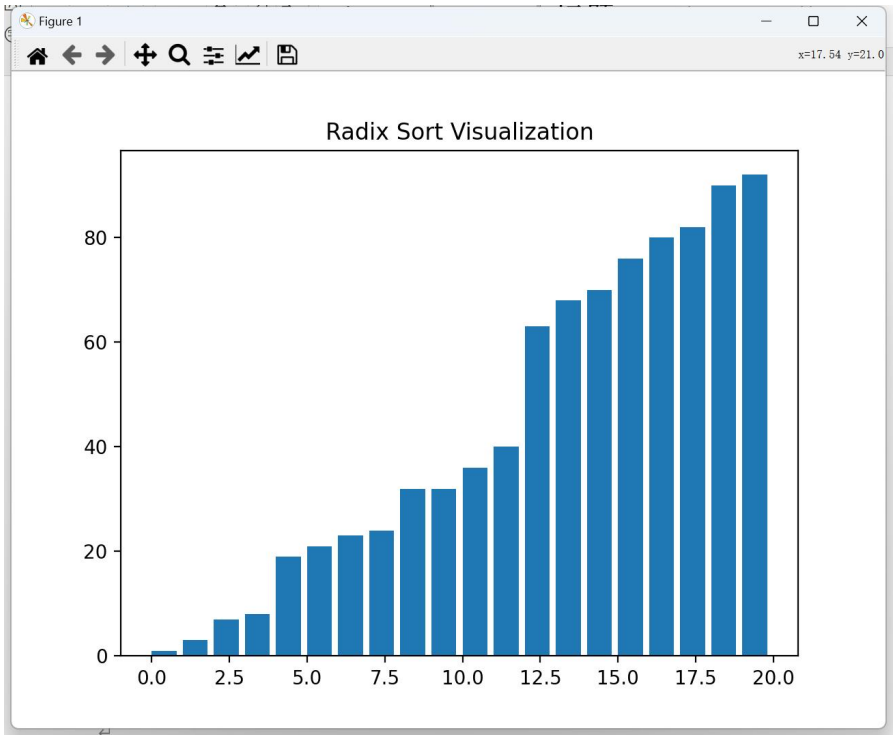

Figure 1
Heap Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Figure 1
Heap Sort Visualization
100
80
60
40
20
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

基数（桶）排序


C:\Users\asus\anaconda3\python.exe "C:\User
基数排序十次平均时间： 0.028099393844604 49

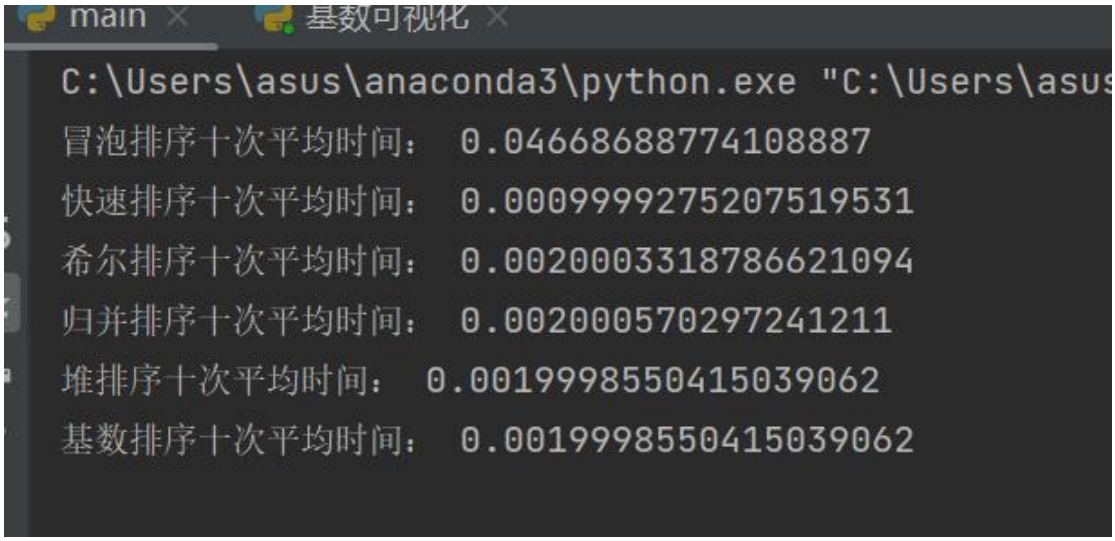

Figure 1
Radix Sort Visualization
0
20
40
60
80
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Figure 1
x=17.54 y=21.0
Radix Sort Visualization
0
20
40
60
80
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

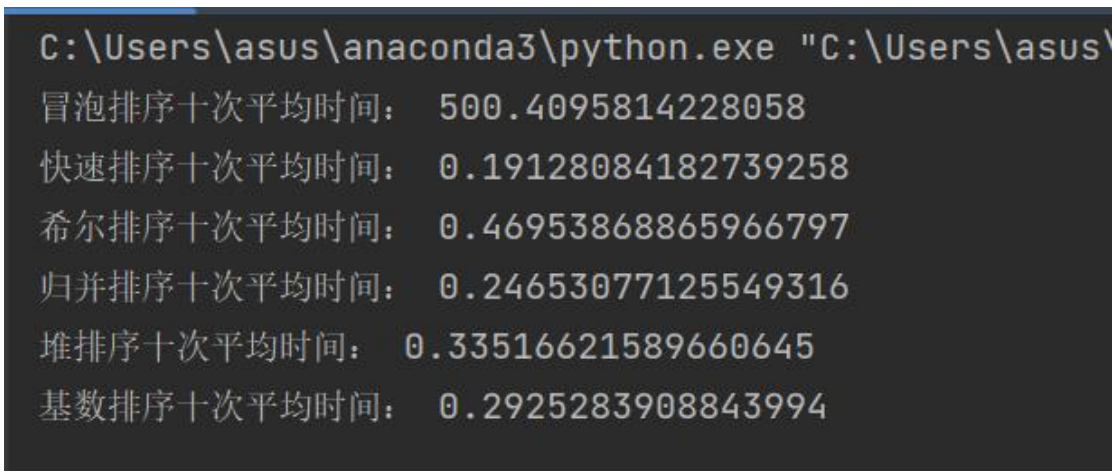
八、总结

实验结果显示：
在排 10000 个数据时：
冒泡(5.4660)>基数(0.0281)>希尔(0.0270)>堆排(0.0257)>归并(0.0229)>快排(0.0150)
这与算法复杂度对应的大致符合，但我们注意到基数排序相较于其他还是过于慢了，考虑到是数据个数问题，因此在此部分替换数据个数重新实验。

数据为 1000 个时：



数据为 100000 个时：



当数据趋于很大时，快排明显优于其他算法，冒泡的时间确实符合 n^2

附录

冒泡：

```
def bubble_sort():
    for i in range(len(random_numbers1)):
        for j in range(len(random_numbers1) - i - 1):
            if random_numbers1[j] > random_numbers1[j + 1]:
                swap(random_numbers1, j, j + 1)
```

快排：

```
def quick_sort(arr):
    if len(arr) == 0 or len(arr) == 1:
        return arr
    random_numbers2 = arr
    piovt_index = 0
    piovt = random_numbers2[piovt_index]
    i = 1
    j = len(arr) - 1
    while (i <= j):
        while i <= j and (random_numbers2[i] <= piovt):
            i += 1
        while i <= j and (random_numbers2[j] >= piovt):
            j -= 1
        if i <= j:
            swap(random_numbers2, i, j)

    swap(random_numbers2, piovt_index, j)
    return quick_sort(random_numbers2[:j]) + [piovt] +
quick_sort(random_numbers2[j + 1:])
```

希尔：

```
var_list = [1, 3, 7, 15, 31, 63, 127, 255, 511, 1023, 2047, 4095]
var_list.reverse()


def insert_sort(arr, start, gap):
    for i in range(start + gap, len(arr), gap):
        position = i
        while position >= gap and arr[position] < arr[position - gap]:
            arr[position], arr[position - gap] = arr[position - gap],
arr[position]
            position -= gap
```

```
def shell_sort():
    for var in var_list:
        for i in range(var):
            insert_sort(random_numbers, i, var)
```

归并：

```
def merge_sort(arr, first, last):
    if first < last:
        mid1 = first + (last - first) // 4
        mid2 = first + 2 * (last - first) // 4
        mid3 = first + 3 * (last - first) // 4

        merge_sort(arr, first, mid1)
        merge_sort(arr, mid1 + 1, mid2)
        merge_sort(arr, mid2 + 1, mid3)
        merge_sort(arr, mid3 + 1, last)

        merge(arr, first, mid1, mid2, mid3, last)


def merge(arr, first, mid1, mid2, mid3, last):
    left_arr = arr[first:mid1 + 1]
    mid1_arr = arr[mid1 + 1:mid2 + 1]
    mid2_arr = arr[mid2 + 1:mid3 + 1]
    mid3_arr = arr[mid3 + 1:last + 1]

    left_arr.append(float('inf'))
    mid1_arr.append(float('inf'))
    mid2_arr.append(float('inf'))
    mid3_arr.append(float('inf'))

    i = j = k = l = 0

    for m in range(first, last + 1):
        if left_arr[i] <= mid1_arr[j] and left_arr[i] <= mid2_arr[k] and
left_arr[i] <= mid3_arr[l]:
            arr[m] = left_arr[i]
            i += 1
        elif mid1_arr[j] <= left_arr[i] and mid1_arr[j] <= mid2_arr[k]
and mid1_arr[j] <= mid3_arr[l]:
            arr[m] = mid1_arr[j]
            j += 1
        elif mid2_arr[k] <= left_arr[i] and mid2_arr[k] <= mid1_arr[j]
```

```
and mid2_arr[k] <= mid3_arr[l]:
            arr[m] = mid2_arr[k]
            k += 1
        else:
            arr[m] = mid3_arr[l]
            l += 1
```

堆排：

```
def heapify(arr, n, i):
    largest = i
    left = 4 * i + 1
    mid1 = 4 * i + 2
    mid2 = 4 * i + 3
    right = 4 * i + 4

    if left < n and arr[left] > arr[largest]:
        largest = left

    if mid1 < n and arr[mid1] > arr[largest]:
        largest = mid1

    if mid2 < n and arr[mid2] > arr[largest]:
        largest = mid2

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    for i in range((n-2) // 4, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
```

基数排序：

```python
def counting_sort(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    for i in range(n):
        index = arr[i] // exp % 10
        count[index] += 1

    for i in range(1, 10):
        count[i] += count[i - 1]

    i = n - 1
    while i >= 0:
        index = arr[i] // exp % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1
        i -= 1

    for i in range(n):
        arr[i] = output[i]


def radix_sort(arr):
    maxnum = max(arr)
    exp = 1
    while maxnum // exp > 0:
        counting_sort(arr, exp)
        exp *= 10
```